

WellNest

Smart Health & Fitness Companion

An Integrated Full-Stack Health & Wellness Ecosystem

Prepared by:

Team C

Aakash G

Shinmaya KB

Manideep Bogireddy

Bhanu Sathvik Vallabhapurapu

Tanguturi Rithika

Under the guidance of:

Mr. Imamuddin Shaik

Infosys
Springboard
April 16, 2026

This document provides a comprehensive overview of the WellNest application including its purpose, architecture, development lifecycle, methodology, and results.

Abstract

The modern digital landscape has seen a significant surge in health-conscious individuals seeking technological interventions to manage their physical and mental well-being. However, a critical observation of the current market reveals a state of extreme "Data Fragmentation." Most users are forced to navigate multiple, disconnected applications—one for tracking steps, another for logging caloric intake, and yet another for monitoring sleep patterns. This lack of integration leads to a disjointed user experience where holistic health insights are nearly impossible to derive. Furthermore, many existing platforms fail to provide robust security for sensitive physiological data, nor do they offer a personalised environment that adapts to a user's specific biological metrics. In response to these challenges, this project introduces WellNest, a comprehensive full-stack web application designed to serve as a unified "Health Ecosystem." The primary objective of WellNest is to centralise fragmented health data into a single, secure, and user-centric platform. By leveraging a modern tech stack (Frontend, Backend, and Database), the application provides a seamless flow of information that allows for real-time tracking and long-term health analytics. The current phase of development focuses on the foundational pillar of the system: Authentication and Fitness Profile Management. This module is the intended solution to the dual problems of data security and lack of personalisation. By implementing secure cryptographic standards for user access and a dynamic onboarding process to capture metrics such as BMI, activity levels, and fitness goals, WellNest establishes a robust data foundation. This ensures that all subsequent modules—including workout tracking, wearable API integration, and AI-driven dashboards—operate within a highly personalised and protected context. Ultimately, WellNest aims to bridge the gap between raw health data and actionable wellness insights, providing users with a clear, consolidated path toward their fitness goals.

Contents

1	Introduction	1
1.1	Background	1
1.2	The Problem: The "Health-as-a-Work" Gap	1
1.3	Proposed Solution: The "Socialised" Health Nest	2
1.3.1	Personalisation through Identity	2
1.3.2	Gamification and Social Integration	2
1.4	Project Objectives	2
2	Literature Review	4
2.1	Introduction to Digital Health Ecosystems	4
2.2	Existing Fitness and Wellness Applications	4
2.3	Security and Privacy in Health Applications	5
2.4	Gamification and Social Influence in Wellness Platforms	5
2.5	Need for an Integrated Health Ecosystem	6
2.6	Research Gap and Project Justification	6
3	Software Requirements Analysis	7
3.1	Functional Requirements	7
3.2	Non-Functional Requirements	9
3.3	System Requirements	9
3.3.1	Software Requirements	10
3.3.2	Hardware Requirements	10
4	Software Design	11
4.1	Software Development Lifecycle	11
4.2	UML Diagrams	11
4.3	Use Case Diagram	12
4.3.1	Sequence Diagrams	13
4.3.2	Class Diagram	16

5	Methodology	18
5.1	Phase 1: Authentication and User Foundation	18
5.2	Phase 2: Health Engine and Dashboard	19
5.3	Phase 3: Tracking and Analytics	20
5.4	Phase 4: Community and Trainer Integration	21
5.5	Summary	22
6	Results	23
6.1	Backend API Endpoints	24
6.2	Health Plan Calculation Results	25
6.3	Frontend Page Coverage	25
6.4	Chatbot Performance	26
6.5	Security Outcomes	26
7	Conclusion	27
7.1	Summary	27
7.2	Key Achievements	27
7.3	Limitations and Future Work	28
7.4	Final Remarks	28

Chapter 1

Introduction

1.1 Background

In recent years, the digital health landscape has shifted from simple pedometers to complex data-tracking ecosystems. While technology has made it easier to record health metrics, it has inadvertently created a "chore-like" environment for users. Most individuals view their health routines as a series of tasks—logging meals, counting steps, and monitoring sleep—leading to high abandonment rates. WellNest is designed to shift this paradigm by transforming health management from a solitary "job" into a personalised, social experience.

1.2 The Problem: The "Health-as-a-Work" Gap

The primary challenge in the current market is the lack of emotional and social engagement in fitness applications, creating a disconnect we identify as the "Health-as-a-Work" gap. Current systems often impose a high cognitive load, treating health tracking as a tedious data-entry obligation rather than a seamless lifestyle choice. This issue is compounded by clinical, impersonal interfaces that present users with cold statistics instead of relatable milestones, failing to connect on a personal level. Furthermore, the isolation factor inherent in most solo-use apps ignores the critical role of social validation and community support in driving human behaviour. Consequently, without social features or interactive elements to sustain interest, the initial user motivation inevitably fades, leading to high abandonment rates within the first few weeks of usage.

1.3 Proposed Solution: The "Socialised" Health Nest

WellNest provides a full-stack solution that prioritises the user's emotional connection to their health. Our approach moves away from rigid tracking and toward a community-centric ecosystem.

1.3.1 Personalisation through Identity

The solution begins with a robust Authentication and Fitness Profile module. Unlike basic sign-up forms, this module establishes a "Wellness Persona." By capturing personal health goals and biological metrics early, the system tailors the entire user experience to feel like a personalised assistant rather than a generic logbook.

1.3.2 Gamification and Social Integration

To ensure health doesn't feel like a job, WellNest integrates:

- **Socialising Features:** Enabling users to connect with peers, share progress, and participate in community-driven health blogs.
- **Trainer Engagement:** A direct bridge to professional trainers, ensuring that expert advice is always part of the user's social circle.
- **Interactive Feedback:** Replacing boring spreadsheets with a Health Analytics Dashboard that visualises progress in an encouraging, easy-to-digest format.

1.4 Project Objectives

The core objectives for the WellNest project are:

- To build a secure and personalised onboarding experience (Module 1).
- To implement a social health ecosystem where users can learn and share via blogs and trainer suggestions.
- To reduce user friction by integrating external health APIs for automated tracking.
- To foster a long-term wellness habit by focusing on community and social interaction.

Chapter 2

Literature Review

2.1 Introduction to Digital Health Ecosystems

The rapid evolution of digital technology has significantly transformed the healthcare and fitness industry. Over the past decade, numerous web and mobile applications have emerged to assist users in tracking physical activity, calorie intake, sleep cycles, and overall wellness. These platforms aim to improve lifestyle habits through self-monitoring and data-driven insights. However, despite technological advancements, most digital health systems operate in isolation, resulting in fragmented user experiences and limited interoperability between applications.

Research in digital health systems emphasises the importance of centralised health ecosystems that integrate multiple data streams into a unified platform. Studies indicate that integrated systems improve user retention, increase engagement, and provide more meaningful long-term insights compared to standalone tracking applications.

2.2 Existing Fitness and Wellness Applications

Several commercial fitness platforms such as MyFitnessPal, Fitbit, and Nike Training Club have gained widespread adoption. These applications offer features including step tracking, calorie logging, workout guidance, and wearable device integration. While they provide valuable services, most focus primarily on quantitative tracking rather than holistic engagement.

Existing systems often rely heavily on manual data entry, which increases cognitive load and reduces long-term adherence. Furthermore, many platforms lack strong social integration

mechanisms, which behavioural psychology studies identify as crucial for habit formation and sustained motivation.

Another limitation observed in current solutions is data fragmentation. Users frequently rely on multiple applications simultaneously—one for workouts, another for nutrition, and another for sleep monitoring. This scattered ecosystem prevents comprehensive health analytics and creates inconsistency in user experience.

2.3 Security and Privacy in Health Applications

Health-related data is highly sensitive and requires strong security mechanisms. Literature on healthcare information systems emphasises the importance of cryptographic standards, secure authentication protocols, and protected data storage. Many early-generation fitness applications stored user credentials with minimal encryption, leading to data breaches and privacy concerns.

Modern secure systems implement password hashing algorithms, token-based authentication, and secure API communication protocols. Research supports the adoption of JSON Web Tokens (JWT) and encrypted storage mechanisms to protect user identity and ensure secure session management. These practices are now considered fundamental in full-stack health application development.

2.4 Gamification and Social Influence in Wellness Platforms

Behavioural science research suggests that social validation and gamification significantly improve adherence to health routines. Applications that incorporate leaderboards, achievement badges, social sharing, and peer interaction demonstrate higher user engagement compared to purely data-centric platforms.

Studies in persuasive technology indicate that community-driven engagement increases intrinsic motivation. Users are more likely to maintain consistency when their progress is visible within a social environment. However, many existing applications implement gamification superficially without deeply integrating social interaction into the system architecture.

2.5 Need for an Integrated Health Ecosystem

Based on existing literature, three major gaps are identified:

- Lack of centralised data integration across fitness domains.
- Insufficient personalisation based on individual health metrics.
- Limited emotional and social engagement mechanisms.

These limitations highlight the need for a unified full-stack system that combines secure authentication, personalised fitness profiling, social connectivity, and scalable architecture.

WellNest addresses these gaps by creating a centralised platform where authentication, profile management, and future modules such as workout tracking and wearable integration operate under a secure and personalised framework. Unlike isolated fitness trackers, WellNest is designed as a scalable ecosystem capable of evolving into a comprehensive digital wellness hub.

2.6 Research Gap and Project Justification

The review of existing systems demonstrates that while technological tools for fitness tracking are abundant, few systems combine security, personalisation, and social engagement within a single architecture. There remains a gap between raw health data collection and meaningful wellness experience design.

WellNest is proposed to bridge this gap by integrating secure authentication mechanisms with personalised fitness profiling and community-based engagement strategies. The system's modular and scalable design allows future incorporation of artificial intelligence analytics and wearable device synchronisation, thereby aligning with emerging trends in digital health innovation.

Chapter 3

Software Requirements Analysis

Requirements analysis is a critical process that defines what the WellNest system must accomplish to ensure effective health monitoring and personalised fitness management. Requirements are categorised into functional and non-functional requirements, along with system requirements that define the necessary hardware and software environment. This analysis ensures that the application is reliable, scalable, and capable of delivering meaningful health insights to users.

3.1 Functional Requirements

Functional requirements specify the core operations and services that the WellNest system must provide. These requirements define how users interact with the system and how different modules collaborate to deliver personalised health management features.

The system is designed to support secure authentication, user profile management, health plan generation, activity tracking, analytics visualisation, and community interaction. Each function contributes to building a comprehensive ecosystem that supports user wellness and engagement.

The following table presents the functional requirements of the WellNest system.

Table 3.1: Functional Requirements for WellNest

ID	Requirement	Description
FR001	User Registration and Login	The system must allow users to register, log in, and authenticate securely using JWT or OAuth.
FR002	Profile Management	Users must be able to create and update their personal and health-related information.
FR003	Health Plan Generation	The system should generate personalised diet and fitness plans based on user data.
FR004	Activity Tracking	Users should be able to log daily activities such as calories, sleep, water intake, and steps.
FR005	Workout Tracking	The system must allow users to record workouts with details such as sets, reps, and duration.
FR006	Meal Tracking	Users should be able to log meals along with nutritional information.
FR007	Dashboard Visualisation	The system should display user progress using charts and analytics dashboards.
FR008	Blog and Community Interaction	Users must be able to create, view, and interact with blog posts.
FR009	Trainer Matching	The system should recommend trainers based on user goals.
FR010	Chatbot Assistance	The system must provide automated responses to user queries.

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the WellNest system, ensuring that it performs efficiently and reliably under various conditions. These requirements address system performance, security, scalability, usability, and maintainability.

The system must ensure secure data handling, fast response times, and a user-friendly interface. It should also support scalability to accommodate a growing number of users and data.

Table 3.2: Non-Functional Requirements for WellNest

ID	Requirement	Description
NFR001	Performance	The system should provide fast response times for user interactions and data retrieval.
NFR002	Scalability	The system must support increasing numbers of users and data without performance degradation.
NFR003	Security	All user data must be securely stored and transmitted using encryption mechanisms.
NFR004	Reliability	The system should ensure consistent performance with minimal downtime.
NFR005	Usability	The interface should be intuitive and easy to use for all users.
NFR006	Maintainability	The system should allow easy updates and modifications.

3.3 System Requirements

System requirements specify the software and hardware environment needed to deploy and run the WellNest application.

3.3.1 Software Requirements

Frontend: React.js is used to build the user interface and manage user interactions.

Backend: Spring Boot framework is used to implement business logic and REST APIs.

Database: MongoDB Atlas is used for storing user data and application records.

Authentication: JWT and OAuth 2.0 are used for secure user authentication.

Libraries and Tools: Axios for API communication, Chart.js for data visualisation, and Maven for dependency management.

3.3.2 Hardware Requirements

- A computer or laptop with minimum 8GB RAM for development and testing.
- Internet connectivity for accessing cloud services and APIs.
- A modern web browser for running the frontend application.

Chapter 4

Software Design

Software design translates the analysed requirements into a structured architecture for the WellNest system. This chapter describes the development lifecycle and presents UML-based representations to illustrate system interactions and workflows.

4.1 Software Development Lifecycle

The development of WellNest follows an incremental and modular approach. The frontend and backend are developed in parallel to ensure efficient progress and early integration.

The backend services are implemented using iterative development cycles, where authentication, profile management, and health plan generation are built and tested independently. In parallel, the frontend components are developed using React and tested with mock API data.

After individual modules reach stability, integration is performed by connecting frontend components with backend APIs. This phase focuses on data consistency, API communication, and performance optimisation. Subsequent iterations refine the system by fixing issues, enhancing features, and improving user experience.

4.2 UML Diagrams

UML diagrams are used to represent system structure and behaviour. These diagrams provide a clear understanding of interactions between users and system components.

4.3 Use Case Diagram

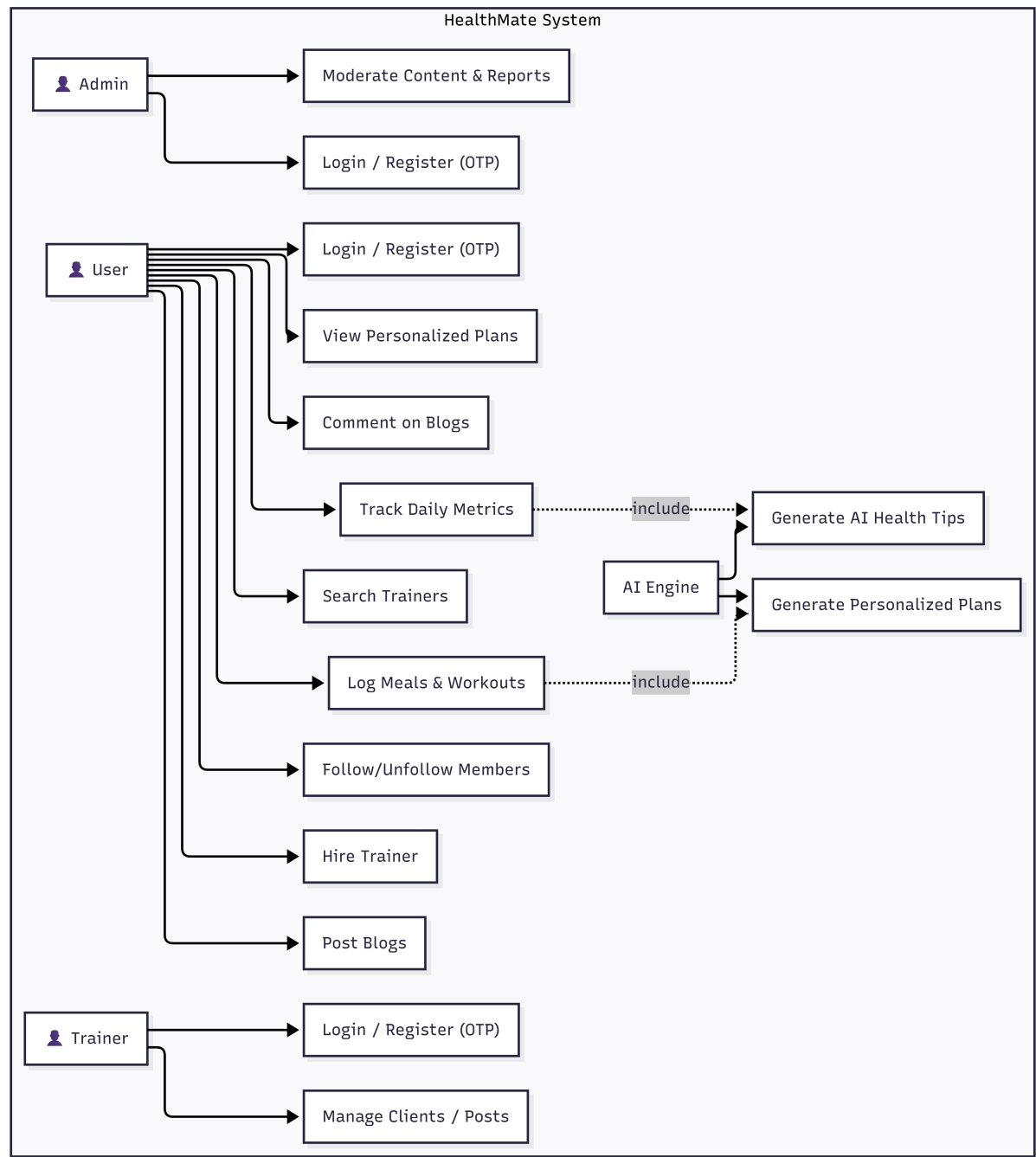


Figure 4.1: Use Case Diagram of WellNest System

The use case diagram of the **WellNest System** illustrates the interactions between different actors and the system’s core functionalities. The primary actors in the system are **User**, **Trainer**, **Admin**, and the **AI Engine**, each performing specific roles.

The **User** is the central actor and interacts with most of the system features. Users can register or log in using an OTP-based authentication mechanism. Once authenticated, they can track daily

health metrics such as steps, water intake, and sleep, as well as log meals and workouts. Based on this data, users can view personalised health plans generated by the system. Additionally, the platform provides social features where users can post blogs, comment on posts, and follow or unfollow other members. Users can also search for trainers and hire them for professional guidance.

The **AI Engine** plays a supportive role by enhancing user experience through intelligent recommendations. It generates AI-based health tips and personalised plans. These functionalities are integrated into the system using «include» relationships, meaning that whenever a user logs daily metrics or meals and workouts, the system automatically triggers AI-driven insights.

The **Trainer** actor can register or log in to the system and manage their clients and related posts. Trainers interact with users through the hiring feature, enabling them to provide personalised guidance and services.

The **Admin** is responsible for maintaining the platform's integrity. Admins can log in and moderate content and reports, ensuring that inappropriate or harmful content is handled effectively.

Overall, the use case diagram provides a high-level view of the system's functionality, highlighting how different actors interact with various features while emphasising the role of AI in delivering personalised health insights.

4.3.1 Sequence Diagrams

Sequence diagrams are used to represent the time-ordered interaction between different components of the WellNest system. These diagrams illustrate how data flows between the frontend, backend services, database, and external systems during key operations.

4.3.1.0 Authentication Flow (OTP-Based)

This sequence diagram represents the secure authentication mechanism implemented in WellNest using OTP-based verification. The process begins when the user enters their email or registration details in the frontend interface. The frontend sends a request to the authentication controller, which generates a one-time password (OTP) through the OTP service.

The OTP service triggers the email service to send the generated code to the user's registered email address. Upon receiving the OTP, the user enters the code into the application. The

frontend forwards this code to the backend for validation.

The authentication controller verifies the OTP and either retrieves an existing user record or creates a new one in the database. Once validation is successful, a JSON Web Token (JWT) is generated and returned to the frontend, which stores it securely and redirects the user to the dashboard.

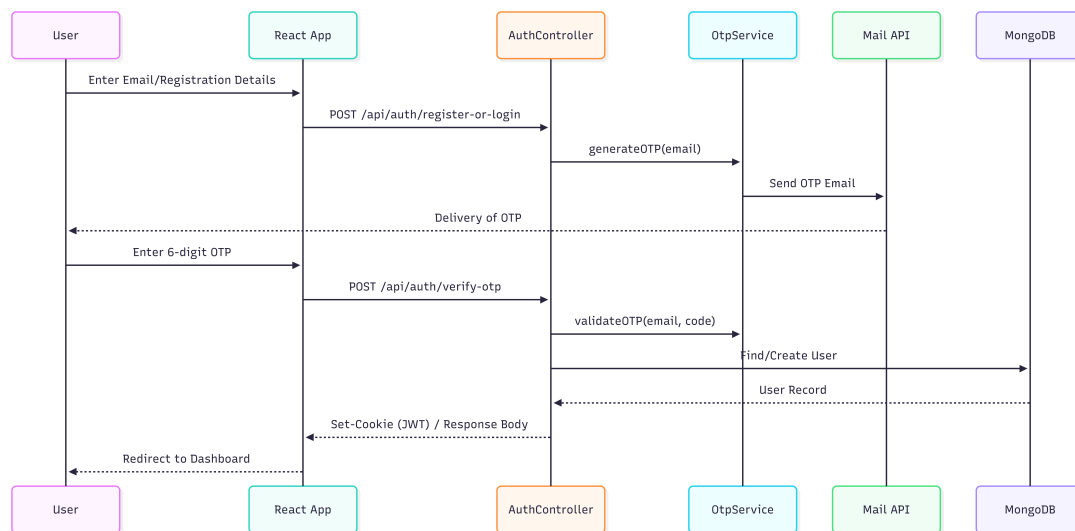


Figure 4.2: Sequence diagram for OTP-based authentication

4.3.1.0 Activity Tracking and AI Tip Generation

This sequence diagram illustrates how user activity logging triggers intelligent feedback within the system. The process begins when the user logs a health metric, such as water intake, through the frontend interface.

The frontend sends the data to the appropriate controller endpoint, which forwards it to the service layer for processing. The service updates the user's daily log in the database and retrieves relevant historical data.

The processed data is then sent to an AI-based analysis module, which evaluates the user's progress and generates personalised feedback. The AI returns suggestions, such as hydration reminders or fitness tips, based on user behaviour patterns.

The service layer sends the updated data along with the AI-generated recommendation back to the frontend, which displays the feedback to the user in real time. This mechanism enhances user engagement by providing actionable insights immediately after data entry.

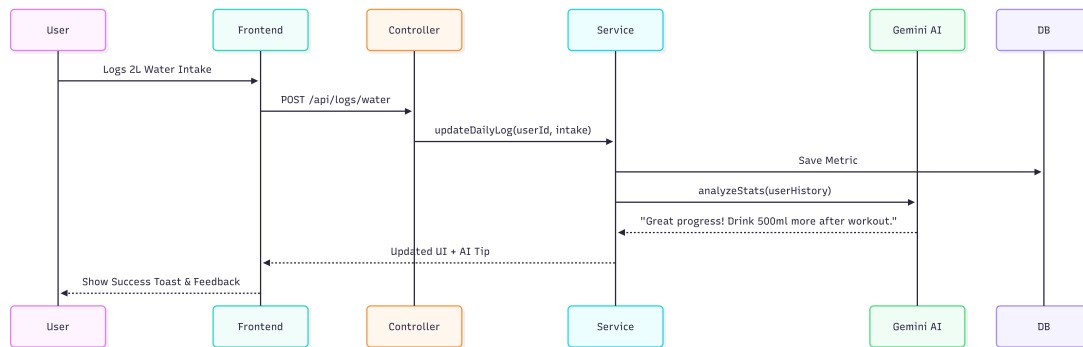


Figure 4.3: Sequence diagram for activity tracking and AI tip generation

4.3.1.0 Moderation Workflow

This sequence diagram describes the moderation process for handling inappropriate content within the community platform. The process is initiated when a user reports a blog post through the frontend interface.

The frontend sends the report request to the moderation controller, which stores the report in the database with a pending status. Administrators or trainers can access the moderation centre to review reported content.

The frontend retrieves the list of pending reports from the backend and displays them to the administrator. Upon reviewing a report, the administrator can take appropriate action, such as approving content removal.

The moderation controller updates the report status in the database and performs the required action, such as deleting the flagged post. A confirmation response is then sent back to the administrator interface.

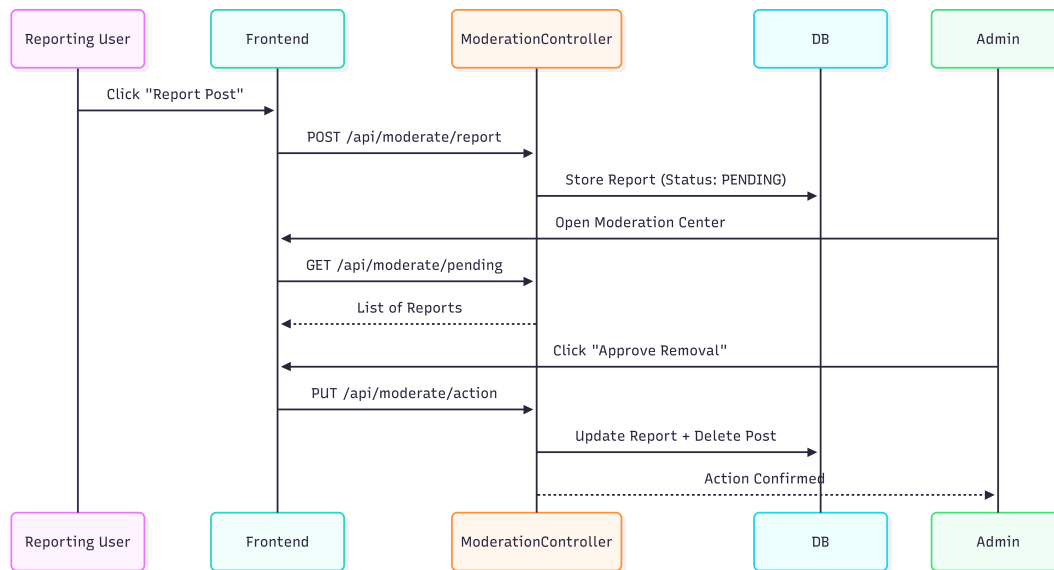


Figure 4.4: Sequence diagram for moderation workflow

4.3.2 Class Diagram

The class diagram of the **WellNest System** represents the static structure of the application by illustrating the key entities, their attributes, and the relationships between them. It provides a conceptual view of how data is organised and how different components interact within the system.

The central entity in the system is the **User** class, which stores essential information such as username, email, roles, and biometric attributes like height and weight. It also includes derived functionality such as BMI calculation, which is used for personalised health plan generation. Each user can have one or more roles, represented by the **Role** class, which is further defined using the **ERole** enumeration containing predefined roles such as USER, TRAINER, and ADMIN. This structure enables role-based access control throughout the application.

The **Trainer** class extends the user's functionality by representing professional profiles. It includes attributes such as specialty, bio, location, and associated reviews, allowing users to connect with certified trainers based on their fitness goals.

The system also includes the **DailyLog** class, which captures user-specific health metrics on a daily basis. These logs store data such as steps, water intake, sleep duration, and calories burned. Each daily log is associated with a specific user and acts as the foundation for analytics and progress tracking.

The **Meal** class is linked to the DailyLog entity and represents individual food entries consumed by the user. It contains nutritional details such as calories and protein, enabling detailed dietary tracking and comparison against personalised targets.

The **BlogPost** class supports the community feature of the platform, allowing users to create and share content. Each blog post contains attributes such as title, content, author reference, and timestamp. Additionally, blog posts are associated with comments, enabling user interaction and engagement within the platform.

The relationships between these classes define the overall system behaviour. A user can have multiple roles, create multiple daily logs, and author multiple blog posts. Each daily log can contain multiple meal entries, while each blog post can have multiple comments. The optional relationship between User and Trainer reflects that only certain users act as trainers.

Overall, the class diagram provides a clear blueprint of the WellNest data model, ensuring consistency, scalability, and efficient interaction between different modules of the system.

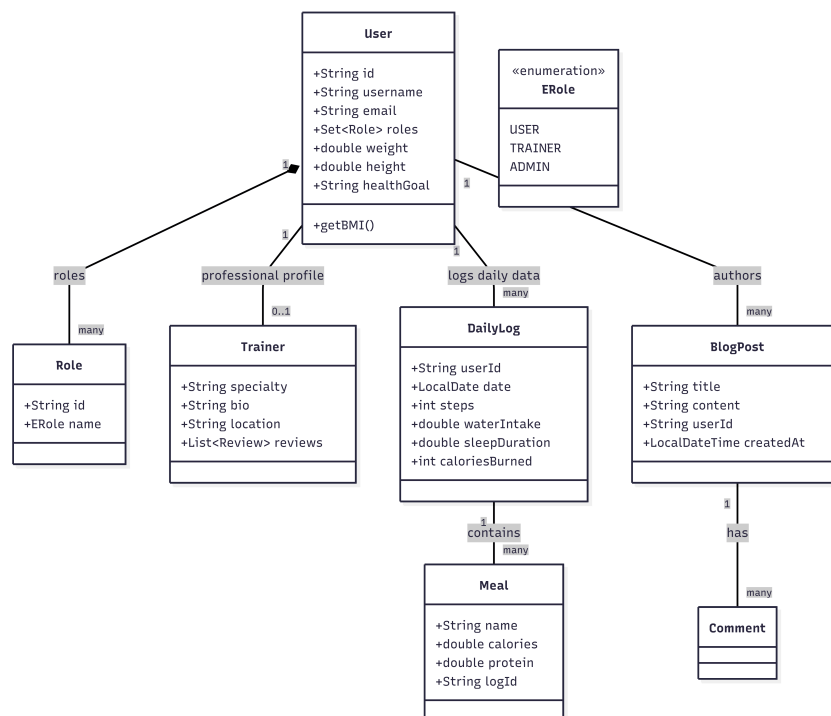


Figure 4.5: Class Diagram of WellNest System

Chapter 5

Methodology

The development of the **WellNest** system was carried out in a structured and incremental manner using a milestone-based approach. The project was divided into four major phases, each focusing on a specific set of functionalities, ensuring modular development, scalability, and continuous improvement.

5.1 Phase 1: Authentication and User Foundation

The first phase focused on establishing a secure and robust user management system. This included implementing user registration and login using OTP-based email verification. JWT (JSON Web Token) authentication was used to maintain secure sessions, while passwords were encrypted using BCrypt.

Additional features such as Google OAuth2 login and password recovery mechanisms were also integrated. Role-based access control was introduced, allowing differentiation between User, Trainer, and Admin roles. This phase ensured a strong foundation for secure access and identity management within the system.

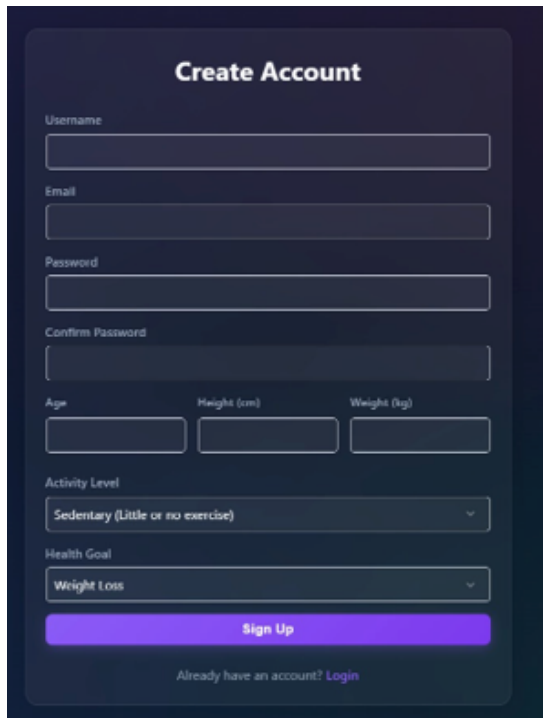
A dark-themed 'Create Account' form. It includes input fields for Username, Email, Password, and Confirm Password. Below these are three input fields for Age, Height (cm), and Weight (kg). There is a dropdown menu for Activity Level with 'Sedentary (little or no exercise)' selected, and another dropdown for Health Goal with 'Weight Loss' selected. A prominent purple 'Sign Up' button is at the bottom, with a link 'Already have an account? Login' below it.

Figure 5.1: User Registration with OTP Verification

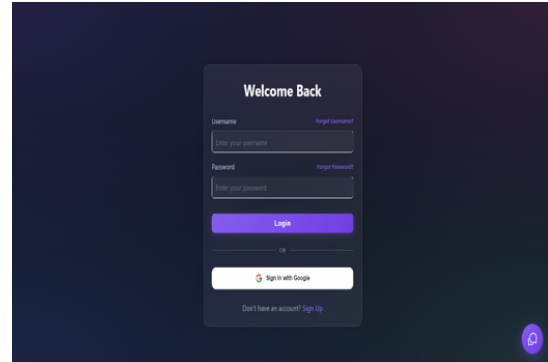
A dark-themed 'Welcome Back' login form. It features input fields for Username and Password, each with a 'Forgot Password?' link. A purple 'Login' button is centered below the fields. Below the button is a link 'Sign in with Google' and a link 'Don't have an account? Sign Up'. A small profile icon is visible in the bottom right corner.

Figure 5.2: Login and Role-Based Access

5.2 Phase 2: Health Engine and Dashboard

The second phase concentrated on building the core intelligence of the application. A health engine was developed to compute user-specific metrics such as BMI and Total Daily Energy Expenditure (TDEE) using standard formulas.

Based on user inputs such as age, weight, height, and fitness goals, the system generated personalised calorie and macronutrient targets. A dynamic dashboard was implemented to display analytics including calorie tracking, goal progress, and streak monitoring. Additionally, meal recommendations were designed using a rotating logic to ensure variety and consistency in user experience.

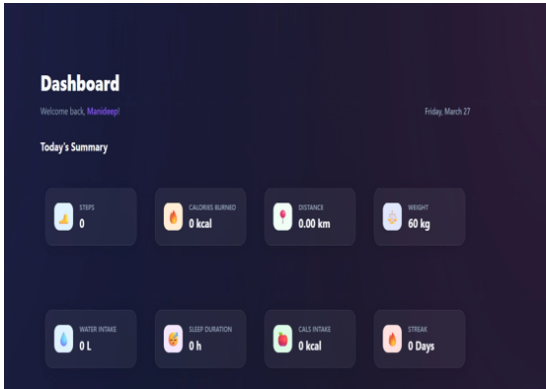


Figure 5.3: User Dashboard Interface

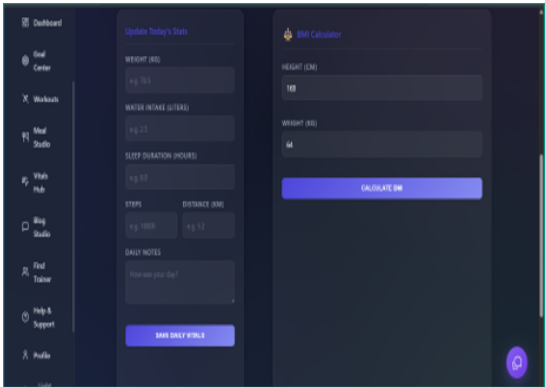


Figure 5.4: BMI and TDEE Calculation Module

5.3 Phase 3: Tracking and Analytics

In this phase, detailed tracking functionalities were implemented to allow users to log their daily activities. This included workout tracking (type, duration, and calories burned), meal tracking with nutritional values, and vitals monitoring such as weight, water intake, sleep, and step count.

An analytics module was introduced to visualise user progress through charts, heatmaps, and activity summaries. The system was designed to maintain continuity of user data, ensuring that previously recorded values are preserved even if daily logs are missed. This phase enhanced user engagement by providing meaningful insights into their health journey.

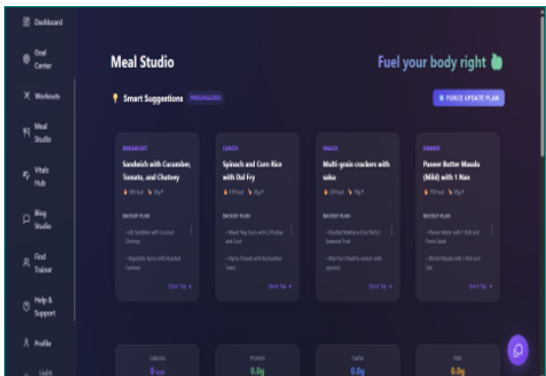


Figure 5.5: Meal and Activity Tracking Interface

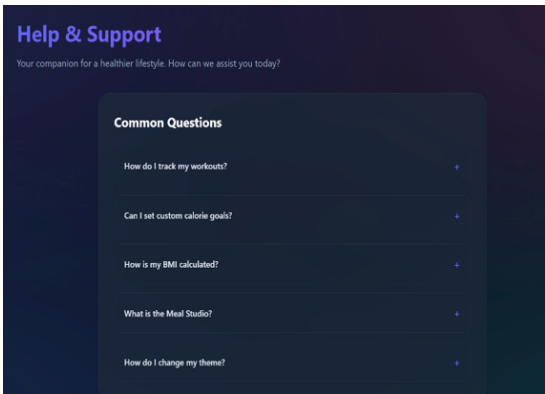


Figure 5.6: Analytics and Support Dashboard

5.4 Phase 4: Community and Trainer Integration

The final phase focused on incorporating social and collaborative features into the system. A blogging platform was developed where users can create posts, interact through comments, and engage with the community.

A moderation system was implemented where admins can review and approve or reject content, ensuring platform safety. Trainer matching functionality was introduced, allowing users to search and connect with trainers based on location and fitness goals.

Additionally, features such as follow/unfollow, likes, and nested comments were implemented to enhance user interaction. A rule-based chatbot was also integrated to assist users in navigating the application. This phase transformed the system into a comprehensive and interactive health ecosystem.

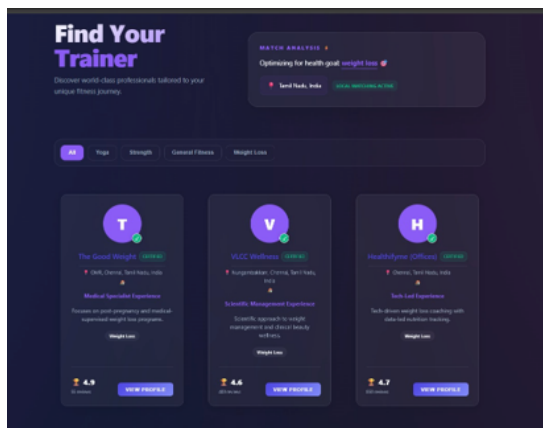


Figure 5.7: Trainer Search and Hiring Module

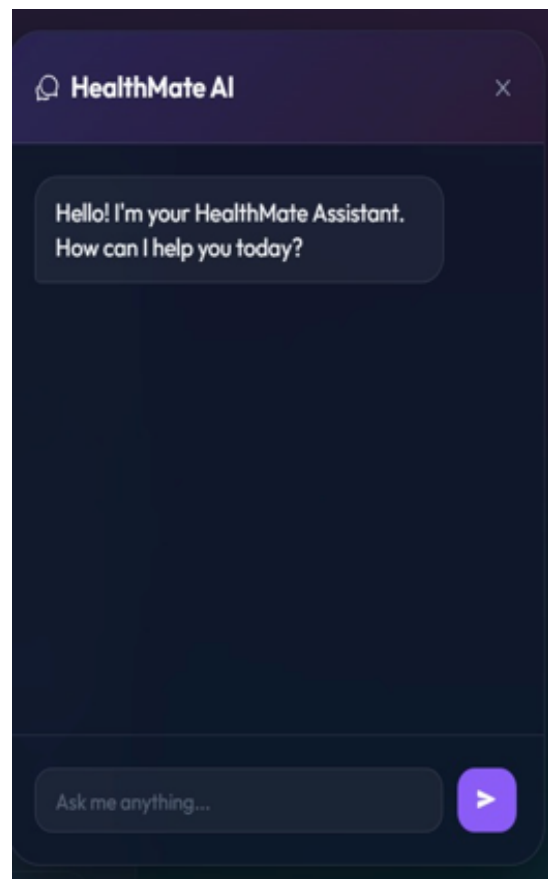


Figure 5.8: AI Chatbot Interface

5.5 Summary

The milestone-based methodology enabled systematic development of the WellNest system, starting from secure authentication to intelligent health analysis and finally to community-driven engagement. This phased approach ensured better testing, modular implementation, and scalability of the application.

Chapter 6

Results

Implemented Features Summary

The completed WellNest application delivers the following functional capabilities, showcasing full-stack integration across authentication, tracking, analytics, and community features. This table summarizes implementation status for key modules.

Feature	Implementation	Status	Backend	Frontend
User registration	email OTP	Login	logout	JWT
Google OAuth 2.0	SSO	Forgot username	password	email
Biometric profile management	BMI calculation and classification	Personalised health plan generation	Daily vitals logging	Workout tracking
Meal tracking with macro breakdown	Goal Centre targets	view	Analytics dashboard with charts	Activity heatmap
Blog Studio	create			

Table 6.1: Feature Implementation Status

Feature	Backend	Frontend
User registration (email + OTP)	✓	✓
Login / logout (JWT)	✓	✓
Google OAuth 2.0 SSO	✓	✓
Forgot username / password (email)	✓	✓
Biometric profile management	✓	✓
BMI calculation and classification	✓	✓
Personalised health plan generation	✓	✓
Daily vitals logging	✓	✓
Workout tracking	✓	✓
Meal tracking with macro breakdown	✓	✓
Goal Centre (targets view)	✓	✓
Analytics dashboard with charts	✓	✓
Activity heatmap	✓	✓
Blog Studio (create / edit posts)	✓	✓
Blog moderation system	✓	✓
Like, comment, and follow features	✓	✓
Content reporting	✓	✓
Trainer profile and matching	✓	✓
In-app AI chatbot	✓	✓
Help Centre (FAQ)	✗	✓
Role-based access (User/Admin/Trainer)	✓	✓

6.1 Backend API Endpoints

A total of 12 REST controllers expose the following endpoint groups:

Table 6.2: REST API Controller Summary

Controller	Base Path	Access
AuthController	/api/auth	Public
UserController	/api/user	Authenticated
AnalyticsController	/api/analytics	Authenticated
WorkoutController	/api/workout	Authenticated
WorkoutPlanController	/api/workoutplan	Authenticated
MealController	/api/meal	Authenticated
DietPlanController	/api/dietplan	Authenticated
BlogController	/api/blog	Authenticated
ModerationController	/api/moderation	Admin/Trainer
FollowController	/api/follow	Authenticated
TrainerController	/api/trainer	Mixed
ChatController	/api/chat	Public
HealthTipController	/api/healthtip	Authenticated

6.2 Health Plan Calculation Results

The personalised health plan engine correctly generates plans across a range of user profiles. For example:

Table 6.3: Sample Plan Outputs for Different User Profiles

Profile	Goal	Daily kcal	Protein (g)	BMI
70 kg, 175 cm, 25 yr, Moderate	Weight Loss	≈1,700	≈126	22.9 (Normal)
80 kg, 180 cm, 30 yr, Active	Muscle Gain	≈2,800	≈176	24.7 (Normal)
60 kg, 165 cm, 28 yr, Light	Maintenance	≈1,850	≈96	22.0 (Normal)

6.3 Frontend Page Coverage

The React SPA implements 17 distinct pages/routes covering the full user journey from registration through daily tracking, community engagement, and trainer discovery. The application

is fully protected via `PrivateRoute` with role-based guards for the Moderation Centre.

6.4 Chatbot Performance

The rule-based chatbot in `ChatService` handles over 30 distinct intent categories covering all major application features. Off-topic queries (politics, history, programming) are blocked with a polite redirect. The chatbot is accessible via a floating UI element on all authenticated pages, requiring no additional login.

6.5 Security Outcomes

- All passwords are BCrypt-hashed with no plain-text storage.
- JWT tokens expire after 24 hours and are validated on every request.
- Role-based method security (`@EnableMethodSecurity`) protects admin and trainer routes.
- Google OAuth 2.0 is integrated without storing OAuth tokens in the database.
- OTP-based email verification prevents spam account creation.

Chapter 7

Conclusion

7.1 Summary

WellNest successfully delivers a comprehensive, free, and open-source health and fitness platform that addresses the fragmentation and cost barriers present in existing solutions. By combining intelligent plan generation, multi-dimensional daily tracking, a moderated community blog, trainer matching, and an in-app chatbot, it provides a holistic health companion in a single deployable application.

The project demonstrates a well-structured three-tier architecture with clean separation of concerns: a React 19 SPA frontend, a Spring Boot 3.2 REST API, and a MongoDB Atlas cloud database. The security model, based on JWT and OAuth 2.0, meets modern web application standards, while the role-based access system ensures community content quality and platform safety.

7.2 Key Achievements

- A scientifically grounded, automated health plan generator using Mifflin-St Jeor BMR, TDEE, and ISSN macro guidelines.
- A fully functional blog ecosystem with moderation workflow, including PENDING/APPROVED/REJECTED states and audit logging.
- A dual authentication system (JWT + Google OAuth 2.0) with OTP email verification.

- Rich analytics visualisations using Chart.js and an activity heatmap.
- A scoped, health-specific in-app chatbot requiring no third-party AI API.

7.3 Limitations and Future Work

While WellNest meets its stated objectives, several areas present opportunities for future enhancement:

1. **AI-powered chatbot:** Replace the keyword-matching engine with a fine-tuned large language model for more natural, context-aware conversations.
2. **Wearable integration:** Add REST webhook support for syncing data from Fitbit, Garmin, or Apple Health APIs.
3. **Mobile application:** Develop a React Native counterpart for iOS and Android to enable push notifications and offline logging.
4. **Dietitian consultation:** Extend the trainer ecosystem to include certified nutritionists with video consultation booking.
5. **Gamification:** Introduce achievement badges, streaks, and leaderboards to increase user retention and engagement.
6. **Production hardening:** Restrict CORS to the production frontend URL, move all secrets to environment variables, and enable HTTPS.
7. **Automated testing:** Expand unit and integration test coverage with JUnit 5 and Mockito for all service layers.

7.4 Final Remarks

WellNest demonstrates that a well-architected, full-stack web application can deliver personalised health guidance at scale without relying on expensive third-party services or proprietary hardware. The project establishes a solid foundation that can be progressively enhanced towards a production-grade health platform serving a broad and diverse user community.